Ingegneria del Software — Soluzione del Tema 14/01/2023

Esercizio 1

Si consideri la seguente classe `SpeedLimits` per memorizzare i limiti di velocità per i vari segmenti (`RoadSegments`) di una strada. La strada inizia al km 0 e ha una lunghezza di `len()` km. Il costruttore assegna lo stesso limite di velocità all'intera strada. Un segmento (`RoadSegment`) è definito come un tratto di strada che inizia e termina con un cambio di limite di velocità e all'interno del quale il limite di velocità non cambia. Il primo segmento inizia al km 0 e l'ultimo segmento termina al km `len()` (escluso).

```
public class SpeedLimits {
  // Considera una strada di lunghezza len e imposta a speedLimit il limite
  // di velocita' per l'intera strada.
  public SpeedLimits(int speedLimit, int len);

  // Ritorna la lunghezza della strada in km.
  public /*@ pure @*/ int len();

  // Ritorna il limite di velocita' al km specificato.
  public /*@ pure @*/ int getSpeedLimitAt(int km);

  // Imposta il limite di velocita' a speedLimit per il tratto di strada che inizia
  // al km start (incluso) e finisce al km end (escluso).
  // Il limite di velocita' deve essere positivo e inferiore o uguale a 130 km orari.
  // Lancia una InvalidBoundariesException se gli estremi del tratto di strada non sono validi.
  public void changeSpeedLimit(int start, int end, int speedLimit) throws InvalidBoundariesException;

  // Ritorna la lista dei segmenti contenuti nella strada. I segmenti sono
  // ritornati in ordine (da quello che inizia per primo a quello che inizia per ultimo).
  public /*@ pure @*/ List<RoadSegment> getSegments();
}

public /*@ pure @*/ RoadSegment {
  // Ritorna il km di inizio del segmento (incluso nel segmento).
  public /*@ pure @*/ int getStart();

  // Ritorna il km di fine del segmento (escluso dal segmento).
  public /*@ pure @*/ int getEnd();

  // Ritorna il limite di velocita' definito per il segmento di strada.
  public /*@ pure @*/ int getSpeedLimit();
}
```

Domanda a)

Specificare in JML il metodo `changeSpeedLimit`.

Soluzione

```
//@ requires speedLimit > 0 && speedLimit <= 130
//@
//@ ensures start >= 0 && end <= len() && start < end &&
//@ (\forall int i; i>=0 && i<len(); getSpeedLimitAt(i) ==
//@    (i>=start && i<end) ? speedLimit : \old(getSpeedLimitAt(i)) )
//@
//@ signals (InvalidBoundariesException e) && (start < 0 || end > len() || start >= end) &&
//@ (\forall int i; i>=0 && i<len(); getSpeedLimitAt(i) == \old(getSpeedLimitAt(i)))
public void changeSpeedLimit(int start, int end, int speedLimit) throws InvalidBoundariesException;
```

Domanda b)

Specificare in JML il metodo `getSegments()`.

Soluzione

```
//@ ensures \result != null && !\result.isEmpty() &&
//@
//@ (\forall RoadSegment s; ; \result.contains(s) <==> (s.getStart() < s.getEnd() &&
//@    (s.getStart() == 0 || getSpeedLimitAt(s.getStart()) != getSpeedLimitAt(s.getStart())-1) &&
//@    (s.getEnd() == len() || getSpeedLimitAt(s.getEnd()-1) != getSpeedLimitAt(s.getEnd())) &&
//@    (\forall int i; i>=s.getStart() && i<s.getEnd()-1;
//@        getSpeedLimitAt(i) == getSpeedLimitAt(i+1) ) ) ) &&
//@ (\forall int i; 0 <= i < \result.size()-1; \result.get(i).getEnd() == \result.get(i+1).getStart())
public /*@ pure @*/ List<RoadSegment> getSegments();
```

Domanda c)

Si consideri la seguente implementazione della classe `SpeedLimits` che usa una mappa per definire i limiti di velocità e un intero per definire la lunghezza della strada. La mappa ha come chiave il km al quale inizia un segmento di strada e come valore il limite di velocità definito per quel segmento di strada.

```
public class SpeedLimits {
  private Map<Integer, Integer> segments;
  private int len;
... }
```

Per tale implementazione, si forniscano l'invariante di rappresentazione e la funzione di astrazione.

Soluzione

L'invariante di rappresentazione richiede che la lunghezza della strada sia positiva. La mappa non deve essere nulla, non deve contenere chiavi o valori nulli. Le chiavi devono essere tra 0 (incluso) e len (escluso). I valori devono essere tra 0 (escluso) e 130 (incluso). Deve esistere la chiave 0. Chiavi che definiscono segmenti consecutivi non possono essere associate allo stesso valore.

```
//@ private invariant
//@ len > 0 && segments != null && segments.containsKey(0) &&
//@
//@ (\forall Integer k; segments.containsKey(k);
//@    k != null && k >= 0 && k < len &&
//@    segments.get(k) != null && segments.get(k) > 0 && segments.get(k) <= 130) &&
//@
//@ (\forall Integer k1; segments.containsKey(k1);
//@    (\forall Integer k2; segments.containsKey(k2) &&
//@       !(\exists Integer k3; segments.containsKey(k3) && k1 < k3 && k3 < k2);
//@       segments.get(k1) != segments.get(k2) ) )
```

La funzione di astrazione definisce `len()` e `getSpeedLimitAt` in base a `len` e `segments`. Tutti gli altri metodi sono definiti a partire da questi.

```
//@ private invariant
//@ len() == len &&
//@ (\forall int i; i >= 0 && i < len;
//@    (\max Integer j; segments.containsKey(j) && j<=i;
//@       getSpeedLimitAt(i) == segments.get(j) ) )
```

Domanda d)

Si consideri una classe `GreenSpeedLimits` che aggiunge un metodo `reduceEmissions` che abbassa del 15% tutti i limiti di velocità salvati al fine di ridurre le emissioni. Può tale classe essere definita come sottoclasse di `SpeedLimits` in accordo con il principio di sostituzione?

Soluzione

La classe può essere definita come sottoclasse di `SpeedLimits` perchè aggiunge un metodo senza alterare i metodi esistenti e senza violare alcun invariante pubblico di `SeriesStore`. Lo stesso risultato del metodo aggiunto si potrebbe ottenere con la classe originale, modificando la velocità individualmente per tutti i segmenti.

Esercizio 2

Si consideri la seguente classe `Car` che rappresenta una autovettura con la sua targa, la sua velocità e il suo colore.

```
public class Car {
  private String plateNumber;
  private int speed;
  private String color;
  public Car(String plateNumber) {
    this.platenumber = plateNumber;
    speed = 0;
    color = "black";
  }
  public String getPlateNumber() { return plateNumber; }
  public void accelerate() { speed++; }
  public void decelerate() { speed--; }
  public int getSpeed() { return this.speed; }
  public void paint(String newColor) { this.color = newColor; }
  public String getColor() { return color; }
}
```

Domanda a)

Si sincronizzino opportunamente i metodi della classe in maniera da massimizzare il parallelismo.

Soluzione

Al fine di massimizzare il parallelismo, sincronizziamo separatamente i metodi che accedono a `speed` e quelli che accedono a `color`, usando due opportuni oggetti separati come "lock". Si noti che il metodo `getPlateNumber` non deve essere sincronizzato perchè il `plateNumber` non viene mai cambiato.

```
public class Car {
  private String plateNumber;
  private int speed;
  private String color;
  private Object speedLock, colorLock;
  public Car(String plateNumber) {
    this.platenumber = plateNumber;
    speed = 0;
    color = "black";
    speedLock = new Object();
    colorLock = new Object();
  }
  public String getPlateNumber() { return plateNumber; }
  public void accelerate() { synchronized(speedLock) speed++; }
  public void decelerate() { synchronized(speedLock) speed--; }
  public int getSpeed() { synchronized(speedLock) return this.speed; }
  public void paint(String newColor) { synchronized(colorLock) this.color = newColor; }
  public String getColor() { synchronized(colorLock) return color; }
}
```

Domanda b)

Si aggiunga alla classe `Car` un metodo `public void limitSpeed()` che crea ed avvia un thread parallelo rispetto al chiamante, il quale, in un ciclo infinito (senza bloccare indefinitamente l'oggetto), decelera l'auto (chiamando il metodo `decelerate`) senza mai andare al di sotto di 10 km/h.

Soluzione

All'interno del ciclo infinito del nuovo thread occorre rendere atomico il frammento di codice che decide se decelerare in funzione della velocità attuale (per non rischiare di decelerare al di sotto dei 10 km/h).

```
public void speedLimit() {
  new Thread() {
    public void run() {
      while (true) {
        synchronized(speedLock) {
          if (getSpeed()>10) decelerate();
        }
      }
    }
  }.start();
}
```

Domanda c)

Fare in modo che il metodo paint sospenda il chiamante in attesa che la macchina sia ferma e cambi il colore solo mentre l'auto è ferma.

Soluzione

```
public void paint(String newColor) {
  synchronized(speedLock) {
    while(speed!=0) speedLock.wait();
    synchronized(colorLock) { this.color = newColor; }
  }
}
```

Occorre anche inserire `speedLock.notifyAll()` nei metodi che modificano il valore di `speed`.

Esercizio 3

Si considerino le seguenti dichiarazioni di classi Java.

```
abstract class LibraryItem {
  public abstract String show();
}
class Text extends LibraryItem {
  public String show() { return "Text"; }
  public String show(LibraryItem a) { return this.show() + " and " + a.show(); }
}

class AudioBook extends Text {
  public String show(Text a) { return "Audio with " + a.show(); }
}
```

e il seguente frammento di codice

```
1  AudioBook item1;
2  Text item2;
3  LibraryItem item3;
4  item3 = new LibraryItem();
5  item2 = new AudioBook();
6  item1 = item2;
7  item3 = new Text();
8  System.out.println(item3.show());
9  System.out.println(item3.show(item2));
10 System.out.println(item2.show(item3));
11 item2 = (AudioBook) item3;
12 item1 = (AudioBook) item2;
13 System.out.println(item1.show());
14 System.out.println(item1.show(item2));
15 System.out.println(item1.show(item3));
```

Domanda a)

Si indichino il numero di ciascuna riga del frammento sopra riportato che genera un errore in fase di compilazione o in fase di esecuzione, motivando concisamente la propria risposta.

Soluzione

Errori di compilazione:

- riga 4 (impossibile istanziare una classe astratta);
- riga 6 (item2 ha tipo statico Text, che è un sopratipo del tipo statico di item1, ossia AudioBook);
- riga 9 (item3 ha tipo statico LibraryItem, quindi è visibile solo show(), non show(Text)).
- Errori run-time: la riga 11 (item3 non ha tipo dinamico AudioBook, quindi il casting fallisce).

Domanda b)

Supponendo che tutte le righe che provocano errori in fase di compilazione o di esecuzione siano state rimosse, mostrare per ogni istruzione di stampa quale stringa viene prodotta in output, motivando brevemente la propria risposta. Riportare *sempre* il numero di riga di fianco alla risposta.

Soluzione

- Riga 8: Text
- Riga 10: Text and Text
- Riga 13: Text
- Riga 14: Audio with Text
- Riga 15: Text and Text

Esercizio 4

Si consideri il seguente metodo statico.

```
1  public static void doSomething(int a, int b) {
2    if (a>0 && b>0) {
3      while (a<=2 && b>0) {
4        if (a==1 && b==2) {
5          a++;
6        }
7        b--;
8      }
9    }
10 }
```

Domanda a)

Si definisca il minimo un numero di casi di test (per a, b) necessari e sufficienti per coprire tutte le istruzioni del metodo, se ciò è possibile.

Soluzione

Basta il caso `a==1`, `b==2`.

NB: in questo caso si percorre due volte il ciclo; la seconda iterazione del ciclo è percorsa con `a==2`, `b==1`, terminando con `b==0` per uscire quindi dal while.

Domanda b)

Si definisca il minimo numero di casi di test necessari e sufficienti per coprire tutte le diramazioni (edge coverage) del metodo, se ciò è possibile.

Soluzione

Le diramazioni del while e del secondo if sono già coperte dal caso precedente. Occorre aggiungere un solo caso per coprire la diramazione else del primo if: ad esempio `a==0`, `b` qualunque.

Domanda c)

Si definisca il minimo numero di casi di test necessari e sufficienti per coprire tutte le diramazioni e condizioni (edge and condition coverage) del metodo, se ciò è possibile.

Soluzione

Occorre considerare i seguenti requisiti.

La copertura della condizioni del primo if: manca il caso in cui `b>0` è `false` (ma serve `a>0` `true` per potere esercitare la condizione), quindi ad esempio `a==1`, `b==0`.

La copertura delle condizioni del while: i casi precedenti non coprono il caso `a<=2` `false`, quindi occorre aggiungere ad esempio il caso `a==3`, con `b>0` per entrare nel primo if.

La copertura delle condizioni del secondo if: il primo caso di test `a==1`, `b==2` copre anche il caso `a==1` `false` (perché `a==2` alla seconda iterazione del ciclo). Resta da coprire solo il caso `b==2` `false` (che richiede `a==1` `true`). Se si sceglie `a==1` e `b==3`, questo può sostituire il primo caso di test, in quanto il valore delle variabili diventerà proprio `a==1`, `b==2` dopo un'iterazione.

In conclusione, sono necessari 4 casi. Ad esempio: `a==1` e `b==0`, `a==0` e `b` qualunque, `a==3` e `b` positivo, `a==1` e `b==3`.